

Prompt Injection and Tool-Use Security in LLM Agents: A Comprehensive Survey

Abstract

The rapid evolution of Large Language Model (LLM) agents from passive conversationalists to autonomous entities capable of tool use, planning, and memory retention has introduced a paradigm shift in artificial intelligence security. While these agentic systems unlock unprecedented capabilities in software development, web navigation, and complex decision-making, they simultaneously expand the attack surface to include sophisticated vectors such as indirect prompt injection (IPI), tool poisoning, memory corruption, and protocol-level exploits. This survey provides a comprehensive synthesis of 300 recent academic studies focusing on the security vulnerabilities and defense mechanisms specific to LLM agents. We categorize the threat landscape into distinct methodological families, including input manipulation, supply-chain attacks on tool ecosystems, persistent state poisoning, and multi-agent collusion. Furthermore, we critically analyze emerging defense architectures ranging from runtime enforcement and causal attribution to formal verification and capability-based isolation. Our analysis reveals that traditional LLM safety alignments are insufficient for agentic contexts, where the coupling of reasoning and action creates unique failure modes such as “confused deputy” scenarios and temporal causal takeovers. We conclude by identifying critical open problems in evaluation benchmarking, the trade-off between utility and security, and the necessity for system-level governance frameworks.

1. Introduction

The transition from static Large Language Models (LLMs) to dynamic, agentic systems represents one of the most significant developments in modern artificial intelligence. Unlike their chatbot predecessors, LLM agents are designed to perceive environments, reason over long horizons, maintain persistent memory, and execute actions through external tools and APIs [1], [2]. This autonomy enables applications ranging from autonomous coding assistants and web browsers to complex multi-agent orchestration systems [3], [4]. However, this shift from text generation to action execution fundamentally alters the security posture of AI systems. The primary threat model evolves from generating harmful content to executing harmful actions, including data exfiltration, unauthorized resource access, and system compromise [5], [6].

The core vulnerability in agentic systems stems from the blurring of boundaries between instructions and data. In traditional software, code and data are strictly separated; in LLM agents, both are represented as natural language tokens within a shared context window. This architectural characteristic allows adversaries to embed malicious instructions within untrusted data sources—such as retrieved documents, tool outputs, or web pages—leading to Indirect Prompt Injection (IPI) [7], [8]. When an agent processes this contaminated data, it may inadvertently execute the embedded commands, treating them as high-priority user intents [9], [10]. The severity of this risk is amplified by the agent’s access to powerful tools, where a single successful injection can lead to remote code execution, financial loss, or privacy breaches [11], [12].

Recent literature has identified a diverse array of attack vectors targeting different components of the agentic pipeline. Attacks now target not only the input prompt but also the tool descriptions [13], [14], the agent’s long-term memory [15], [16], and the communication protocols between agents and tools [17], [18]. For instance, FuncPoison demonstrates how poisoning function library descriptions can hijack multi-agent autonomous driving systems [13], while MemMorph shows that

injecting crafted records into long-term memory can stealthily alter agent behavior across sessions [16]. Furthermore, the rise of Model Context Protocol (MCP) and similar standards has introduced new protocol-level vulnerabilities, where attackers can exploit trust boundaries between clients and servers to escalate privileges [19], [20].

Defending against these threats requires moving beyond simple input filtering. The probabilistic nature of LLMs renders deterministic security guarantees difficult to achieve using traditional methods [21]. Consequently, the research community has proposed a wide spectrum of defense mechanisms. These include architectural interventions like Plan-then-Execute paradigms that decouple planning from untrusted data [4], [22], runtime monitoring systems that enforce policy constraints [23], [24], and causal attribution methods that distinguish between user intent and injected noise [25], [26]. Despite these advances, a significant gap remains between theoretical defenses and practical deployment, with many solutions incurring substantial latency overheads or failing against adaptive adversaries [27], [28].

This survey synthesizes the current state of knowledge regarding prompt injection and tool-use security in LLM agents. We organize the field into coherent methodological families, analyzing the mechanics of attacks, the efficacy of defenses, and the limitations of current evaluation benchmarks. By integrating insights from 300 distinct studies, we aim to provide a rigorous foundation for understanding the security challenges of autonomous AI and charting a path toward robust, trustworthy agentic systems.

2. Method

The security of LLM agents is a multifaceted problem spanning the entire lifecycle of agent operation, from tool selection and planning to execution and memory consolidation. We categorize the methodological landscape into several key domains: adversarial attacks on tool ecosystems, indirect prompt injection mechanisms, memory and state poisoning, multi-agent security dynamics, and defensive architectures.

2.1 Adversarial Attacks on Tool Ecosystems and Function Calling

The integration of external tools via function calling interfaces constitutes a primary attack surface for LLM agents. Adversaries exploit the reliance of agents on tool metadata, descriptions, and return values to manipulate behavior without directly altering the user prompt.

2.1.1 Tool Description and Metadata Poisoning A prevalent attack vector involves poisoning the metadata associated with available tools. Since agents rely on natural language descriptions to select appropriate functions, attackers can inject malicious instructions or bias the selection process by modifying these descriptions. FuncPoison introduces a supply-side attack where malicious function definitions containing templated invocation examples are injected into shared function libraries, exploiting the LLM’s structural bias toward valid templates to hijack function selection in autonomous driving systems [13]. Similarly, the Attractive Metadata Attack (AMA) formulates malicious metadata generation as a state-action-value optimization problem, iteratively refining tool names and descriptions to maximize the probability of malicious tool selection while maintaining semantic coherence [29].

The severity of this threat is highlighted by the Model Context Protocol (MCP), a standard for connecting agents to external data sources. MCPTox establishes a benchmark demonstrating that poisoning tool metadata in real-world MCP servers can achieve attack success rates (ASR) as

high as 72.8%, with implicit parameter tampering proving particularly effective [19]. Breaking MCP with Function Hijacking Attacks (FHA) further adapts gradient-based optimization (GCG) to inject adversarial suffixes directly into function descriptions, achieving ASRs between 70% and 98% regardless of user query semantics [14]. These attacks reveal a critical vulnerability: agents often exhibit “blind obedience” to tool descriptions due to LLM sycophancy, enabling high-success preference manipulation [30].

2.1.2 Tool Selection Bias and Retrieval Manipulation Beyond static description poisoning, attackers can manipulate the dynamic retrieval of tools. ToolTweak employs an iterative black-box optimization loop where an attacker LLM proposes modified tool names and descriptions, using the victim agent’s selection rates as feedback to bias behavior across diverse models [31]. In retrieval-augmented settings, ToolFlood formalizes “Top-k Domination,” where injecting a small swarm of semantically optimized tools dominates the retrieval results, effectively hiding valid tools from the agent via semantic covering [32]. This denial-of-visibility attack exploits the geometry of embedding spaces, rendering legitimate tools inaccessible even if they are present in the database.

The vulnerability of tool selection is further quantified by TOOLCERT, which models tool selection as a Bernoulli process and demonstrates that robustness collapses under adaptive attacks targeting retrieval saturation, dropping certified robust accuracy to below 20% after a single round of adaptation [33]. These findings suggest that tool selection is a fragile component of the agentic pipeline, highly susceptible to both semantic persuasion and embedding space manipulation.

2.1.3 Malicious Tool Generation and Execution Attacks also target the generation and execution phases of tool use. MalTool orchestrates coding LLMs to synthesize diverse malicious tools, utilizing execution-based verification to ensure functional correctness against target behaviors, achieving a 100% success rate across tested coding LLMs [34]. In multi-tool environments, Cross-Tool Harvesting and Polluting (XTHP) exploits semantic and syntax dependencies to inject malicious tools into agent control flows, forcing sequential execution and enabling data harvesting [35]. Furthermore, the “Confused Deputy” problem is exploited in Multi-Agent Systems (MAS), where adversarial inputs hijack system control flow by manipulating error handling and task planning metadata, tricking orchestrators into invoking arbitrary malicious code [36].

2.2 Indirect Prompt Injection (IPI) Mechanisms

Indirect Prompt Injection (IPI) remains the most pervasive threat, where malicious instructions are embedded in external data sources rather than the user prompt. The literature reveals a sophisticated evolution of IPI techniques, moving from simple string injection to complex, context-aware, and adaptive strategies.

2.2.1 Injection Vectors and Surfaces IPI attacks leverage various surfaces for injection. Web Indirect Prompt Injection (WIPI) injects crafted natural language instructions into external webpage source code, exploiting the retrieval step to ensure visibility and achieving ASRs up to 93.86% [10]. In coding agents, QueryIPI formalizes query-agnostic IPI, optimizing malicious tool descriptions to attack under arbitrary user queries by leveraging internal system prompts as invariants [9]. The “Surface You Test Is Not the Surface That Breaks” study highlights that vulnerability arises from model-surface pairing interactions, with schema surfaces (tool descriptions) often dominating data surfaces (tool outputs) for specific model families [37].

Hidden comment injection represents a stealthy vector, exploiting the disparity between rendered UI and raw text ingestion. When Skills Lie demonstrates that HTML comments in skill documentation, invisible to human reviewers, can effectively steer agent planning toward sensitive actions [38]. Similarly, ChatInject formats malicious payloads to mimic native chat templates, forging role tags (system, user, assistant) within tool outputs to bypass instruction hierarchies [39].

2.2.2 Adaptive and Optimization-Based Attacks Static injections are increasingly ineffective against robust models, leading to the development of adaptive attacks. AdapTools integrates an adaptive attack strategy construction module employing an iterative LLM-as-Optimizer loop to generate transferable adversarial strategies, doubling the ASR compared to baseline methods [27]. IterInject closes the loop between injection, diagnosis, and refinement, using structured diagnostic feedback to evolve payloads beyond initial strategy spaces, achieving a 47.8% ASR on AgentDojo [40].

Gradient-based optimization techniques, originally developed for jailbreaking, have been adapted for IPI. The Universal Adversarial Trigger Injection via GCG embeds optimized triggers into webpage HTML segments, reliably hijacking agent actions across diverse navigation goals [41]. ObliInject formulates prompt injection as minimizing an order-oblivious loss over random permutations of clean and contaminated segments, yielding robustness to any ordering of input data [42]. These adaptive methods consistently bypass static defenses, with some studies reporting ASRs exceeding 50% against eight tested defense mechanisms [28].

2.2.3 Temporal and Multi-Turn Dynamics IPI is not limited to single-turn interactions. Depth-Dependent Indirect Prompt Injection analysis reveals that injection depth is a primary risk variable, with ASR decaying from 60% at depth 1 to 0% at depth 5 due to context distribution and task-length constraints [43]. However, multi-turn attacks pose significant challenges. A3S-Bench constructs multi-turn dialogues employing temporal fragmentation and spatial concealment, elevating average risk trigger rates from 28.3% to 52.6% [44]. The “Foot-in-the-Door” (FITD) attack injects a harmless distractor request prior to a malicious instruction, establishing a compliance pattern that increases ASR by up to 44.8% [45].

Furthermore, Sleeper Attacks introduce a temporal dimension where adversarial observations persist in agent state without immediate execution, triggering malicious instructions in future interactions. This cross-interaction threat model shows ASRs rising from 11.1% in direct baselines to 39.9% in sleeper scenarios [46]. Hidden in Memory extends this to persistent memory, achieving near-ceiling injection rates (99.8%) where retrieved poisoned memories drive agent actions in subsequent sessions [47].

2.3 Memory and State Poisoning

As agents evolve to maintain long-term memory and persistent state, new attack vectors emerge targeting the storage and retrieval mechanisms themselves. These attacks aim to corrupt the agent’s knowledge base or behavioral policies, leading to persistent vulnerabilities.

2.3.1 Memory Poisoning Mechanisms Memory poisoning attacks exploit the agent’s reliance on stored experience for decision-making. MemMorph executes a three-stage pipeline to inject crafted records into long-term memory, achieving an 85.9% ASR by exploiting the agent’s trust in historical context [16]. Obsessive Experience Poisoning (OEP) constructs “Clean Edge-Cases”

with locally correct but non-transferable solutions, coupled with adversarial consequence triplets to exploit agent loss aversion, inducing self-generated harmful rules via reflection mechanisms [15].

State Contamination in Memory-Augmented Agents reveals that toxic influence propagates via distinct channels: raw transcripts drive overt toxicity, while compressed memory carries sub-threshold influence, creating a “Sub-threshold Propagation Gap” that evades standard detection [48]. Zombie Agents exploit memory evolution functions to commit malicious payloads into long-term storage during benign sessions, using recursive self-replication to defeat FIFO context truncation [49].

2.3.2 Persistent State and Cross-Session Attacks The persistence of agent state introduces cross-session vulnerabilities. Stored Prompt Injection (SPI) formalizes a lifecycle comprising write, incorporation, and activation stages across sessions, transforming prompt injection from an ephemeral threat to a system-level vulnerability embedded in execution state [50]. Your Agent, Their Asset decomposes agent persistent state into Capability, Identity, and Knowledge (CIK), demonstrating that poisoning any CIK dimension increases average ASR from 24.6% to 64-74% [51].

When Routine Chats Turn Toxic identifies “Unintended Long-Term State Poisoning” (ULSP), where benign routine interactions cumulatively corrupt persistent state files, inducing authorization drift and tool-use escalation without explicit attackers [52]. This highlights a critical insight: safety failures can emerge from the accumulation of benign but unverified interactions, challenging the assumption that only malicious inputs pose a risk.

2.4 Multi-Agent System Security

The deployment of multiple interacting agents introduces complex security dynamics, including collusion, propagation, and emergent vulnerabilities.

2.4.1 Propagation and Infectious Attacks Multi-agent systems are susceptible to infectious jailbreaks, where a single compromised agent can propagate malicious behavior across the network. Agent Smith formalizes infectious dynamics using SIR-like differential equations, demonstrating that a single image can jailbreak one million multimodal agents exponentially fast ($O(\log N)$ chat rounds) [53]. Prompt Infection utilizes self-replicating prompts containing hijacking and payload components, enabling LLM-to-LLM propagation and increasing ASR by 209% in global messaging architectures [54].

ClawWorm exploits flat context trust to inject malicious directives into core configuration files, implementing dual-anchor persistence to ensure survival across session restarts and routine interactions [55]. These self-propagating attacks highlight the fragility of trust chains in multi-agent ecosystems, where a single breach can compromise the entire network.

2.4.2 Collusion and Integrity Attacks Integrity attacks target the fairness and reliability of multi-agent collaborations. Demonstrations of Integrity Attacks introduce a taxonomy of four integrity-preserving attacks (Scapegoater, Boaster, Self-Dealer, Free-Rider) that manipulate evaluation metrics and resource allocation without breaking end-task functionality [56]. MAD-SPEAR simulates Sybil agents by replicating output formats to artificially inflate malicious node counts, inverting fault-tolerance factors and forcing infinite debate loops [57].

Agent Cascading Injection (ACIARENA) quantifies cascading failure risk via a Propagation Vulnerability Index (PVI), revealing that role design and interaction patterns critically determine

vulnerability, with some configurations reaching 100% ASR under hijacking [58]. These findings underscore the need for robust consensus mechanisms and trust verification in multi-agent deployments.

2.5 Defensive Architectures and Mitigation Strategies

Defending against agentic threats requires a shift from prompt-level filtering to system-level governance. The literature proposes a diverse array of defensive architectures, ranging from runtime enforcement to formal verification.

2.5.1 Architectural Isolation and Plan-then-Execute Architectural separation is a foundational defense strategy. The Plan-then-Execute (PTE) paradigm decouples planning from execution by generating a task-specific program over trusted APIs before observing runtime content, preventing attackers from altering control flow via injected data [4], [22]. Single-Shot Planning adapts this to GUI environments, generating complete execution graphs with conditional branches prior to interaction, achieving a 19% improvement in pass rates while maintaining security [59].

Agent Privilege Separation splits agents into Reader and Actor components, where the Reader serializes untrusted content into structured JSON and the Actor executes privileged tools only on validated data, eliminating baseline successes in email injection scenarios [60]. Similarly, Sovereign Agentic Loops (SAL) interpose a control plane between stochastic reasoning and execution, validating intents against true system state via an obfuscation membrane [61].

2.5.2 Runtime Enforcement and Policy Control Runtime enforcement mechanisms monitor and restrict agent actions based on predefined policies. Progent intercepts tool calls via a modular wrapper enforcing JSON Schema-based privilege policies, reducing ASR from 41.2% to 2.2% [23]. CLAWGUARD enforces security at tool-call boundaries via content sanitization and rule evaluation, achieving 0% ASR on AgentDojo [24]. SkillGuard implements a dual-plane governance model regulating context influence and action side effects via skill manifests and runtime permission access control [62].

Advanced systems like Cognitive Control Architecture (CCA) enforce control-flow and data-flow integrity via an Intent Graph, detecting trajectory deviations caused by malicious objectives [63]. Policy Compiler for Agentic Systems (PCAS) instruments agents to maintain a dynamic dependency graph, querying a Datalog-based policy engine to enforce causal provenance [64]. These approaches emphasize the principle of least privilege, dynamically restricting tool exposure based on task context [65], [66].

2.5.3 Causal Attribution and Verification Causal methods aim to distinguish between user intent and injected noise by analyzing the causal drivers of agent actions. CausalArmor computes Leave-One-Out attributions to detect causal dominance shifts, triggering targeted sanitization only when untrusted spans drive privileged actions [25]. AttrGuard implements action-level causal attribution via parallel counterfactual tests, achieving 0% ASR on AgentDojo by verifying that actions stem from user intent rather than untrusted observations [26].

Verify-Before-Commit strategies, such as VIGIL, decouple reasoning from execution via a five-stage pipeline including intent anchoring and speculative backtracking, reducing ASR by over 22% compared to dynamic baselines [67]. MELON (Masked re-Execution and Tool comparisON) executes parallel agent trajectories with masked user inputs to isolate injected instructions, achieving an

ASR of 0.24% [68]. These methods provide robust defense without requiring extensive retraining, though they often incur computational overhead.

2.5.4 Memory and State Defense Defending memory requires specialized mechanisms to validate stored information. A-MemGuard intercepts the memory-to-action pipeline via consensus-based validation, generating parallel reasoning paths to flag anomalous deviations [69]. StateGuard intervenes at the post-execution audit stage to prevent unintended long-term state poisoning, reducing harm scores to near zero [52]. Secure Forgetting frameworks enable behavioral forgetting via prompt modification rather than parameter updates, addressing privacy concerns in persistent agents [70].

2.5.5 Multi-Agent and Protocol Defenses In multi-agent settings, Sentinel Agents deploy specialized monitors alongside a coordinator to intercept and flag interactions, achieving 100% detection rates in synthetic corpora [71]. Guardians framework employs a Reverse Turing Test to detect rogue agents via network scanning and baseline deviation analysis [72]. For protocol-level security, ATTESTMCP extends the Model Context Protocol with cryptographic capability attestation and HMAC-SHA256 message signing to prevent unauthorized tool invocation [18].

2.6 Evaluation Benchmarks and Metrics

Robust evaluation is critical for assessing agentic security. A variety of benchmarks have been developed to capture the complexity of real-world threats.

- **AgentDojo:** A dynamic environment evaluating prompt injection attacks and defenses across stateful simulated environments (Workspace, Slack, Banking), featuring formal utility checks via deterministic state mutation [73].
- **INJECAGENT:** Constructs 1,054 test cases pairing user tools with attacker tools, differentiating between base and enhanced settings to evaluate indirect prompt injection [7].
- **AgentHarm:** Measures harmfulness across 11 categories augmented to 440 tasks requiring sequential tool calls, integrating model capability metrics into harm scoring [5].
- **LivePI:** Deploys agents on a real Ubuntu VM with live tools, constructing 169 attack instances across seven surfaces to evaluate production-like vulnerabilities [74].
- **SKILL-INJECT:** Measures agent vulnerability to skill file attacks, constructing 202 injection-task pairs across 23 skills [75].
- **MCPTox:** The first systematic benchmark targeting tool metadata poisoning in real-world MCP ecosystems [19].
- **TAMAS:** Benchmarks adversarial risks in multi-agent systems across six attack vectors, introducing the Effective Robustness Score (ERS) [76].

These benchmarks reveal that frontier models remain vulnerable, with ASRs varying significantly based on the attack surface and defense mechanism employed. For instance, while some defenses achieve near-zero ASR on AgentDojo, they often suffer from high false positive rates or utility degradation [77], [78].

3. Challenges and Outlook

Despite significant progress, the field of LLM agent security faces several critical challenges that hinder the deployment of robust, trustworthy systems.

3.1 The Utility-Security Trade-off

A recurring theme in the literature is the tension between security and utility. Strict enforcement mechanisms, such as rigid policy guards or extensive verification loops, often introduce significant latency and computational overhead. For example, speculative reasoning-verification loops in VIGIL introduce substantial overhead for complex tasks [67], while runtime mediation in AGENTRIM increases latency by 1.85x [65]. Furthermore, overly aggressive defenses can lead to “over-defense,” where legitimate tasks are blocked due to false positives. CommandSans, while effective, is vulnerable to tokenization manipulation, and strict filtering can degrade benign agent performance [79], [80]. Balancing these competing objectives remains an open problem, requiring adaptive mechanisms that scale verification intensity based on risk assessment [81].

3.2 Adaptive Adversaries and Evaluation Gaps

Current defenses often fail against adaptive adversaries who optimize attacks specifically to bypass detection. Studies show that adaptive attacks utilizing optimized adversarial strings consistently bypass all tested defenses with ASRs exceeding 50% [28]. Moreover, evaluation benchmarks often rely on static datasets or simulated environments that may not capture the dynamism of real-world deployments. LivePI highlights that frontier backbones remain vulnerable in production-like environments, particularly via group-chat channels [74]. The lack of standardized, dynamic benchmarks that account for multi-turn, cross-session, and multi-agent interactions limits the generalizability of defense claims [44], [82].

3.3 Systemic and Protocol-Level Vulnerabilities

As agents become more integrated into software ecosystems, systemic vulnerabilities emerge. The “Confused Deputy” problem, where agents trust metadata from sub-agents or external tools, remains a critical flaw [36]. Protocol-level vulnerabilities in standards like MCP amplify attack success rates by 23-41% due to absent capability attestation [18]. Addressing these issues requires a shift from model-centric security to system-centric governance, incorporating principles of least privilege, zero-trust architectures, and formal verification [21], [11].

3.4 Future Research Directions

Future research must focus on several key areas: 1. **Formal Verification:** Developing methods to provide deterministic guarantees for agentic behavior, moving beyond probabilistic classifiers [83], [63]. 2. **Dynamic Governance:** Creating adaptive governance frameworks that adjust security policies in real-time based on context and risk [84], [85]. 3. **Cross-Modal Security:** Extending security analysis to multimodal agents, where visual and auditory inputs introduce new injection vectors [86], [87]. 4. **Human-in-the-Loop:** Designing effective human-in-the-loop mechanisms that enhance security without compromising autonomy, particularly for high-stakes decisions [88], [89]. 5. **Standardization:** Establishing industry-wide standards for tool authentication, capability attestation, and secure communication protocols [18], [20].

4. Conclusion

The security of LLM agents is a complex, evolving challenge that demands a holistic approach spanning model alignment, architectural design, and system governance. This survey has synthesized a vast body of literature demonstrating that while LLM agents offer transformative potential, they introduce unique vulnerabilities such as indirect prompt injection, tool poisoning, and memory

corruption. Traditional LLM safety measures are insufficient; robust defense requires a combination of architectural isolation, runtime enforcement, causal verification, and rigorous evaluation. As the field advances, the integration of formal methods, adaptive governance, and standardized protocols will be essential to building agentic systems that are not only capable but also secure and trustworthy. The path forward lies in recognizing that security is not a feature to be added but a fundamental property to be engineered into the fabric of autonomous AI.

References

- [1] Hang Su, Jun Luo, Chang Liu. (2024). A Survey on Autonomy-Induced Security Risks in Large Model-Based Agents. arXiv:2506.23844.
- [2] Lingbo Mo, Zeyi Liao, Boyuan Zheng. (2024). A Trembling House of Cards? Mapping Adversarial Attacks against Language Agents. arXiv:2402.10196.
- [3] Daniel Jones, Giorgio Severi, Martin Pouliot. (2024). A Systematization of Security Vulnerabilities in Computer Use Agents. arXiv:2507.05445.
- [4] Julien Piet, Annabella Chow, Yiwei Hou. (2024). Web Agents Should Adopt the Plan-Then-Execute Paradigm. arXiv:2605.14290.
- [5] Maksym Andriushchenko, Alexandra Souly, Mateusz Dziemian. (2024). AGENTHARM: A BENCHMARK FOR MEASURING HARMFULNESS OF LLM AGENTS. arXiv:2410.09024.
- [6] Matteo Lupinacci, Francesco Aurelio Pironti, Francesco Blefari. (2025). The Dark Side of LLMs: Agent-based Attacks for Complete Computer Takeover. arXiv:2507.06850.
- [7] Qiusi Zhan, Zhixiang Liang, Zifan Ying. (2024). INJECAGENT: Benchmarking Indirect Prompt Injections in Tool-Integrated Large Language Model Agents. arXiv:2403.02691.
- [8] Kai Greshake, Sahar Abdelnabi, Shailesh Mishra. (2023). Not what you’ve signed up for: Compromising Real-World LLM-Integrated Applications with Indirect Prompt Injection. arXiv:2302.12173.
- [9] Yuchong Xie, Zesen Liu, Mingyu Luo. (2025). QueryIPI: Query-agnostic Indirect Prompt Injection on Coding Agents. arXiv:2510.23675.
- [10] Fangzhou Wu, Shutong Wu, Yulong Cao. (2024). WIPI: A New Web Threat for LLM-Driven Web Agents. arXiv:2402.16965.
- [11] Zonghao Ying, Xiao Yang, Siyang Wu. (2026). Uncovering Security Threats and Architecting Defenses in Autonomous Agents: A Case Study of OpenClaw. arXiv:2603.12644.
- [12] Eunkyu Lee, Donghyeon Kim, Wonyoung Kim. (2025). Takedown: How It’s Done in Modern Coding Agent Exploits. arXiv:2509.24240.
- [13] Yuzhen Long, Songze Li. (2024). FuncPoison: Poisoning Function Library to Hijack Multi-agent Autonomous Driving Systems. arXiv:2509.24408.
- [14] Yannis Belkhiter, Giulio Zizzo, Sergio Maffei. (2025). Breaking MCP with Function Hijacking Attacks: Novel Threats for Function Calling and Agentic Models. arXiv:2604.20994.
- [15] Kaixiang Wang, Jiong Lou, Zhaojiacheng Zhou. (2024). OEP: Poisoning Self-Evolving LLM Agents via Locally Correct but Non-Transferable Experiences. arXiv:2605.18930.

- [16] Xuanye Zhang, Yongsen Zheng, Zhuqin Xu. (2025). MemMorph: Tool Hijacking in LLM Agents via Memory Poisoning. arXiv:2605.26154.
- [17] Mohamed Amine Ferrag, Norbert Tihanyi, Djallel Hamouda. (2025). From Prompt Injections to Protocol Exploits: Threats in LLM-Powered AI Agents Workflows. arXiv:2506.23260.
- [18] Narek Maloyan, Dmitry Namiot. (2024). Breaking the Protocol: Security Analysis of the Model Context Protocol Specification and Prompt Injection Vulnerabilities in Tool-Integrated LLM Agents. arXiv:2601.17549.
- [19] Zhiqiang Wang, Yichao Gao, Yanting Wang. (2025). MCPTox: A Benchmark for Tool Poisoning Attack on Real-World MCP Servers. arXiv:2508.14925.
- [20] Shiva Gaire, Srijan Gyawali, Saroj Mishra. (2025). Systematization of Knowledge: Security and Safety in the Model Context Protocol Ecosystem. arXiv:2512.08290.
- [21] Mihai Christodorescu, Earlene Fernandes, Ashish Hooda. (2024). Systems Security Foundations for Agentic Computing. arXiv:2512.01295.
- [22] Ron F. Del Rosario, Klaudia Krawiecka, Christian Schroeder de Witt. (2025). Architecting Resilient LLM Agents: A Guide to Secure Plan-then-Execute Implementations. arXiv:2509.08646.
- [23] Tianneng Shi, Jingxuan He, Zhun Wang. (2024). Progent: Programmable Privilege Control for LLM Agents. arXiv:2504.11703.
- [24] Wei Zhao, Zhe Li, Peixin Zhang. (2024). CLAWGUARD: A RUNTIME SECURITY FRAMEWORK FOR TOOL-AUGMENTED LLM AGENTS AGAINST INDIRECT PROMPT INJECTION. arXiv:2604.11790.
- [25] Minbeom Kim, Mihir Parmar, Phillip Wallis. (2025). CausalArmor: Efficient Indirect Prompt Injection Guardrails via Causal Attribution. arXiv:2602.07918.
- [26] Yu He, Haozhe Zhu, Yiming Li. (2024). AttrGuard: Defeating Indirect Prompt Injection in LLM Agents via Causal Attribution of Tool Invocations. arXiv:2603.10749.
- [27] Che Wang, Jiaming Zhang, Ziqi Zhang. (2025). AdapTools: Adaptive Tool-based Indirect Prompt Injection Attacks on Agentic LLMs. arXiv:2602.20720.
- [28] Qiusi Zhan, Richard Fang, Henil Shalin Panchal. (2024). Adaptive Attacks Break Defenses Against Indirect Prompt Injection Attacks on LLM Agents. arXiv:2503.00061.
- [29] Kanghua Mo, Li Hu, Yucheng Long. (2024). Attractive Metadata Attack: Inducing LLM Agents to Invoke Malicious Tools. arXiv:2508.02110.
- [30] Yongjian Guo, Puzhuo Liu, Wanlun Ma. (2025). Systematic Analysis of MCP Security. arXiv:2508.12538.
- [31] Jonathan Sneh, Ruomei Yan, Jialin Yu. (2025). TOOLTWEAK: AN ATTACK ON TOOL SELECTION IN LLM-BASED AGENTS. arXiv:2510.02554.
- [32] Hussein Jawad, Nicolas J-B. Brunel. (2025). ToolFlood: Beyond Selection — Hiding Valid Tools from LLM Agents via Semantic Covering. arXiv:2603.13950.
- [33] Jehyeok Yeon, Isha Chaudhary, Gagandeep Singh. (2025). Quantifying Distributional Robustness of Agentic Tool-Selection. arXiv:2510.03992.

- [34] Yuepeng Hu, Yuqi Jia, Mengyuan Li. (2025). MalTool: Malicious Tool Attacks on LLM Agents. arXiv:2602.12194.
- [35] Zichuan Li, Jian Cui, Xiaojing Liao. (2025). Les Dissonances: Cross-Tool Harvesting and Polluting in Multi-Tool Empowered LLM Agents. arXiv:2504.03111.
- [36] Harold Triedman, Rishi Jha, Vitaly Shmatikov. (2025). Multi-Agent Systems Execute Arbitrary Malicious Code. arXiv:2503.12188.
- [37] Shifat E Arman, Syed Nazmus Sakib, Nafiul Haque. (2025). The Surface You Test Is Not the Surface That Breaks. arXiv:2605.30454.
- [38] Qianli Wang, Boyang Ma, Minghui Xu. (2024). When Skills Lie: Hidden-Comment Injection in LLM Agents. arXiv:2602.10498.
- [39] Hwan Chang, Yonghyun Jun, Hwanhee Lee. (2025). CHATINJECT: ABUSING CHAT TEMPLATES FOR PROMPT INJECTION IN LLM AGENTS. arXiv:2509.22830.
- [40] Zixuan Chen, Jiayang Chen, Li Luo. (2025). IterInject: Indirect Prompt Injection Against LLM Agents via Feedback-Guided Iterative Optimization. arXiv:2605.24659.
- [41] Sam Johnson, Viet Pham, Thai Le. (2025). Manipulating LLM Web Agents with Indirect Prompt Injection Attack via HTML Accessibility Tree. arXiv:2507.14799.
- [42] Reachal Wang, Yuqi Jia, Neil Zhenqiang Gong. (2024). ObliInjection: Order-Oblivious Prompt Injection Attack to LLM Agents with Multi-source Data. arXiv:2512.09321.
- [43] Mohammadreza Rashidi. (2024). Depth-Dependent Indirect Prompt Injection in Tool-Calling ReAct Agents: Injection Depth, Payload Framing, and Turn-Budget Sensitivity. arXiv:2605.30686.
- [44] Jianan Ma, Xiaohu Du, Ruixiao Lin. (2026). Benchmarking Autonomous Agents against Temporal, Spatial, and Semantic Evasions. arXiv:2605.22321.
- [45] Itay Nakash, George Kour, Guy Uziel. (2024). Breaking ReAct Agents: Foot-in-the-Door Attack Will Get You In. arXiv:2410.16950.
- [46] Yongxiang Li, Moxin Li, Zhixin Ma. (2026). Plant, Persist, Trigger: Sleeper Attack on Large Language Model Agents. arXiv:2605.28201.
- [47] Sidharth Pulipaka, Stanislaw Hlebik, Leonidas Raghav. (2026). Hidden in Memory: Sleeper Memory Poisoning in LLM Agents. arXiv:2605.15338.
- [48] Yian Wang, Agam Goyal, Yuen Chen. (2025). State Contamination in Memory-Augmented LLM Agents. arXiv:2605.16746.
- [49] Xianglin Yang, Yufei He, Shuo Ji. (2025). Zombie Agents: Persistent Control of Self-Evolving LLM Agents via Self-Reinforcing Injections. arXiv:2602.15654.
- [50] Yuanbo Xie, Tianyun Liu, Yingjie Zhang. (2025). What If Prompt Injection Never Left? Exploring Cross-Session Stored Prompt Injection in Agentic Systems. arXiv:2606.04425.
- [51] Zijun Wang, Haoqin Tu, Letian Zhang. (2026). Your Agent, Their Asset: A Real-World Safety Analysis of OpenClaw. arXiv:2604.04759.
- [52] Xiaoyu Xu, Minxin Du, Qipeng Xie. (2024). When Routine Chats Turn Toxic: Unintended Long-Term State Poisoning in Personalized Agents. arXiv:2605.06731.

- [53] Xiangming Gu, Xiaosen Zheng, Tianyu Pang. (2024). Agent Smith: A Single Image Can Jailbreak One Million Multimodal LLM Agents Exponentially Fast. arXiv:2402.08567.
- [54] Mo Tiwari, Donghyun Lee. (2024). Prompt Infection: LLM-to-LLM Prompt Injection Within Multi-Agent Systems. arXiv:2410.07283.
- [55] Yihao Zhang, Zeming Wei, Xiaokun Luan. (2024). ClawWorm: Self-Propagating Attacks Across LLM Agent Ecosystems. arXiv:2603.15727.
- [56] Can Zheng, Yuhan Cao, Xiaoning Dong. (2025). Demonstrations of Integrity Attacks in Multi-Agent Systems. arXiv:2506.04572.
- [57] Yu Cui, Hongyang Du. (2025). MAD-SPEAR: A Conformity-Driven Prompt Injection Attack on Multi-Agent Debate Systems. arXiv:2507.13038.
- [58] Hengyu An, Minxi Li, Jinghuai Zhang. (2025). ACIARENA: Toward Unified Evaluation for Agent Cascading Injection. arXiv:2604.07775.
- [59] Hanna Foerster, Robert Mullins, Tom Blanchard. (2025). Single-Shot Planning for Secure Computer Use Agents. arXiv:2601.09923.
- [60] Darren Cheng, Wen-Kwang Tsao. (2024). Agent Privilege Separation in OpenClaw: A Structural Defense Against Prompt Injection. arXiv:2603.13424.
- [61] Jun He, Deyang Yu. (2025). Sovereign Agentic Loops: Decoupling AI Reasoning from Execution in Real-World Systems. arXiv:2604.22136.
- [62] Shidong Pan, Xiaoyu Sun, Tianyi Zhang. (2024). SkillGuard: A Permission Framework for Agent Skills. arXiv:2606.03024.
- [63] Zaiye Chen, Tianze Hu, Zhibo Liang. (2025). Cognitive Control Architecture (CCA): A Lifecycle Supervision Framework for Robustly Aligned AI Agents. arXiv:2512.06716.
- [64] Nils Palumbo, Sarthak Choudhary, Jihye Choi. (2025). Policy Compiler for Secure Agentic Systems. arXiv:2602.16708.
- [65] Roy Betser, Shamik Bose, Amit Giloni. (2025). AgenTRIM: Tool Risk Mitigation for Agentic AI. arXiv:2601.12449.
- [66] Yifeng Cai, Ziming Wang, Zhaomeng Deng. (2025). Who Grants the Agent Power? Defending Against Instruction Injection via Task-Centric Access Control. arXiv:2510.26212.
- [67] Junda Lin, Zhaomeng Zhou, Zhi Zheng. (2025). VIGIL: Defending LLM Agents Against Tool Stream Injection via Verify-Before-Commit. arXiv:2601.05755.
- [68] Kaijie Zhu, Xianjun Yang, Jindong Wang. (2025). MELON: Provable Defense Against Indirect Prompt Injection Attacks in AI Agents. arXiv:2502.05174.
- [69] Qianshan Wei, Tengchao Yang, Yaochen Wang. (2025). A-MemGuard: A Proactive Defense Framework for LLM-Based Agent Memory. arXiv:2510.02373.
- [70] Dayong Ye, Tianqing Zhu, Congcong Zhu. (2024). Secure Forgetting: A Framework for Privacy-Driven Unlearning in Large Language Model (LLM)-Based Agents. arXiv:2604.00430.
- [71] Diego Gosmar, Deborah A. Dahl. (2025). Sentinel Agents for Secure and Trustworthy Agentic AI in Multi-Agent Systems. arXiv:2509.14956.

- [72] Saikat Barua, Mostafizur Rahman, Rafiul Islam. (2024). Guardians: Preventing Jail-Break Prompts in Multi-Agent Systems. arXiv:2502.16750.
- [73] Edoardo Debenedetti, Jie Zhang, Mislav Balunovic. (2024). AgentDojo: A Dynamic Environment to Evaluate Prompt Injection Attacks and Defenses for LLM Agents. arXiv:2406.13352.
- [74] Lei Zhao, Abhay Bhaskar, Edgar Dobriban. (2026). LivePI: More Realistic Benchmarking of Agents Against Indirect Prompt Injection. arXiv:2605.17986.
- [75] David Schmotz, Luca Beurer-Kellner, Sahar Abdelnabi. (2025). SKILL-INJECT: Measuring Agent Vulnerability to Skill File Attacks. arXiv:2602.20156.
- [76] Ishan Kavathekar, Hemang Jain, Ameya Rathod. (2025). TAMAS: Benchmarking Adversarial Risks in Multi-Agent LLM Systems. arXiv:2511.05269.
- [77] Yuxu Ge. (2026). Governance Architecture for Autonomous Agent Systems: Threats, Framework, and Engineering Practice. arXiv:2603.07191.
- [78] Rishika Bhagwatkar, Kevin Kasa, Abhay Puri. (2025). Indirect Prompt Injections: Are Firewalls All You Need, or Stronger Benchmarks?. arXiv:2510.05244.
- [79] Debeshee Das, Luca Beurer-Kellner, Marc Fischer. (2025). CommandSans: SECURING AI AGENTS WITH SURGICAL PRECISION PROMPT SANITIZATION. arXiv:2510.08829.
- [80] Sechan Lee, Sangdon Park. (2025). AI Kill Switch for Malicious Web-based LLM Agents. arXiv:2511.13725.
- [81] Xixun Lin, Yang Liu, Yancheng Chen. (2026). SAFEHARNESS: Lifecycle-Integrated Security Architecture for LLM-based Agent Deployment. arXiv:2604.13630.
- [82] Hao Li, Ruoyao Wen, Shanghao Shi. (2025). AgentDyn: A Dynamic Open-Ended Benchmark for Evaluating Prompt Injection Attacks of Real-World Agent Security System. arXiv:2602.03117.
- [83] Zac Garby, Andrew D. Gordon, David Sands. (2024). The LLMbda Calculus: AI Agents, Conversations, and Information Flow. arXiv:2602.20064.
- [84] Bronislav Sidik, Lior Rokach. (2026). Beyond Static Sandboxing: Learned Capability Governance for Autonomous AI Agents. arXiv:2604.11839.
- [85] Shaona Ghosh, Barnaby Simkin, Kyriacos Shiarlis. (2025). A Safety and Security Framework for Real-World Agentic Systems. arXiv:2511.21990.
- [86] Xiaohan Fu, Zihan Wang, Shuheng Li. (2024). MISUSING TOOLS IN LARGE LANGUAGE MODELS WITH VISUAL ADVERSARIAL EXAMPLES. arXiv:2310.03185.
- [87] Yijie Lu, Tianjie Ju, Manman Zhao. (2024). EVA: Red-Teaming GUI Agents via Evolving Indirect Prompt Injection. arXiv:2505.14289.
- [88] Author One, Author Two, Author Three. (2026). PRUDENTIA: Optimizing Autonomy in Secure AI Agents via Policy-Aware Planning. arXiv:2602.11416.
- [89] Avishag Shapira, Parth Atulbhai Gandhi, Edan Habler. (2024). Mind the Web: The Security of Web Use Agents. arXiv:2506.07153.